```
    this.elementWithTextTarget.innerText = this.newText()
  }
}
```

Now the `flipState()` method automatically causes `visibleValueChanged` to be invoked because it changes the value of `visibleValue`. Inside that method we adjust the target and text. What's nice about this mechanism is that it will keep the target and text in sync with the value no matter where in the controller we change it—if we change the value outside the controller, we still have to separately manage the other changes.

An especially cool feature of using the `changed` mechanism is that it operates even if the DOM data element is changed directly. You can show this in the JavaScript console. Reload the page, then in the JavaScript console type the following:

```
> fav = fav = document.querySelector("[data-controller=favorite-toggle]")
  <THE ELEMENT>
> fav.dataset.favoriteToggleVisibleValue = "false"
  "False"
```

When you enter the second line, Stimulus will notice the DOM change, trigger the `visibleValueChanged` method, and hide the Favorites section. What's nice about this ability is that it means that Stimulus controllers can cascade effects just by changing data attributes in other elements. This gives us very reactive behavior, where we can redraw the DOM automatically in response to the state of the data changing.

Any `changed` method that you have in the controller is automatically called when the controller is initialized using either the initial value given in the markup or the default, which is why we were able to remove our `connect` method and still get our startup behavior.

The changed method can take two optional arguments, the first of which is the new value that is being changed to, and the second of which is the previous value that is being changed from, `visibleValueChanged(new, old)`. Note that the value has already changed by the time the callback is called.